

Design and Implementation of Epidemic Big Data Visualization Platform Based on Vue + ECharts

Jianhua Xie*, Zhongming Yang, Zhang Jun, Zhao Xi, Yifang Tang

Computer Engineering Technical College
Guangdong Polytechnic of Science and Technology
Zhuhai, China

*Corresponding author: jmwisdom@163.com

Abstract—Data visualization technology makes massive data more intuitive and easy to analyze. Based on the epidemic data from the National Bureau of Statistics of China, with the help of ECharts chart, elementUI component library and Vue technology, the data are visualized by using visualization technology and map integration. Through node. JS, Express The framework and MySQL technology realize the annual data management, regional data management and user management of the epidemic situation, display the epidemic situation of each region from multiple perspectives, and provide users with a reliable and convenient understanding channel and data management platform. It provides convenience for people to understand the data of the new coronavirus epidemic, analyze the development trend of the epidemic and manage the big data of the epidemic.

Keywords—Data Visualization, Vue Framework, ECharts chart, ElementUI component, Data Management

I. INTRODUCTION

Data Visualization is a data analysis method to mine valuable information contained in massive data in the form of graphs and charts, which provides a more intuitive display method for data analysis [1], and its advantage is that the efficiency of reading and understanding data has been greatly improved [2]. Through visual epidemic data and intuitive epidemic form display, it not only enables people to better grasp the trend of the epidemic and enhance people's attention to the epidemic, but also plays a positive role in promoting the traceability, monitoring and deployment of epidemic prevention and control.

Enterprise Charts, a top Apache Foundation project, is a free and open source visualization library implemented in JavaScript [3]. It is compatible with most of the current common browsers, runs smoothly across platforms and terminals, and has hundreds of billions of data visualization rendering capabilities [4]. Its bottom layer relies on the lightweight Canvas library Zrender, which provides rich visualization types, such as maps, thermal maps, line charts, histograms, word clouds, etc., for geographic data visualization. It also supports the mutual collocation of various graphics [5], supports DataFrame, Dict and other formats, supports asynchronous rendering of data block loading, and provides interactive functions such as legends, prompts and animation effects, which can fully meet the needs of front-end data visualization.

Epidemic data processing and visualization use HTML, CSS, ElementUI framework to realize page layout and epidemic information list display, use JavaScript language combined with real-time datinterface and JQuery plug-in to obtain real-time epidemic data, realize data structuring and page data rendering. Finally, the map is generated by Echarts and processed by visual mapping function. This paper uses the diversified charts of Echarts and the real-time epidemic data of the National Bureau of Statistics of China to vividly show the content of the big data of epidemic information [6].

II. SYSTEM DESIGN

A. System architecture design

The system uses B/S architecture to avoid client installation, which makes the iterative upgrade of the system faster and easier [7]. As shown in Figure 1, the overall architecture of the system is divided into three layers: application layer, service layer and data layer. The data layer contains the detailed database of epidemic situation (user information table, epidemic situation data table of 31 provinces, etc.), and MYSQL is used for storage [8-9]. The service layer contains the web page framework and data services that the platform relies on, sends requests, passes parameters, returns data from the back-end database, stores information and API interface files (Echarts Server) through the HTTP interface of Axios technology. (ECharts-Server). The front-end framework uses VUE technology, and the back-end uses Express framework, which responds to front-end requests and passes data to the front-end in JSON format. Express is a flexible Node. JS Web application development framework, which keeps the minimum scale and uses various HTTP utilities and middleware. Quickly and easily create powerful APIs that provide a robust set of features for Web and mobile applications [10]. The top layer is the application layer, which realizes the visualization of national epidemic data, epidemic data analysis and user information with the support of the classic modular front-end framework of ElementUI and the rich and diverse chart element library and map element library in Echarts. The application layer is divided into two modules: epidemic data visualization based on VUE + ECHARTS and epidemic data management system based on VUE + ELEMENTUI + ECHARTS.

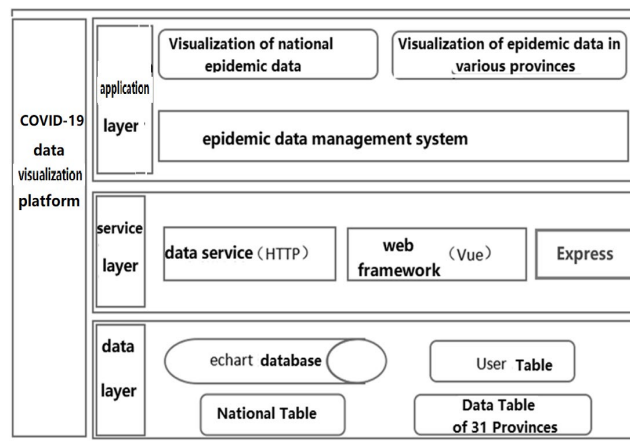


Figure 1. System Architecture Diagram

The front end of the system is based on the data visualization technology of Echarts, through the establishment of line charts, histograms, pie charts and other ways, with the help of multi-chart components provided by Echarts technology, it can intuitively and clearly analyze and display the visualization of epidemic data in various

provinces and regions in China. The back-end of the system uses Node + Express + MySQL to establish a local database based on the data of the national epidemic official website. The API interface is designed through the Express framework to allow the front-end to obtain data. The system design is shown in Figure 2.

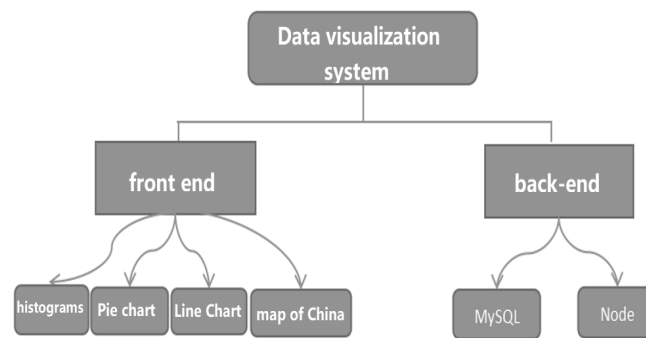


Figure 2. System Design Diagram

B. System process design

The system flow is shown in Figure 3. Front-end users can enter the visual display interface of epidemic data, view the epidemic data of the corresponding province, and switch the epidemic data of the corresponding province by clicking on the map.

After logging in, the back-end administrator enters the epidemic data management system, including role management, regional epidemic data management and annual epidemic data management. The platform data is managed by collecting real-time epidemic data from the official website, so as to achieve the purpose of updating data visualization.

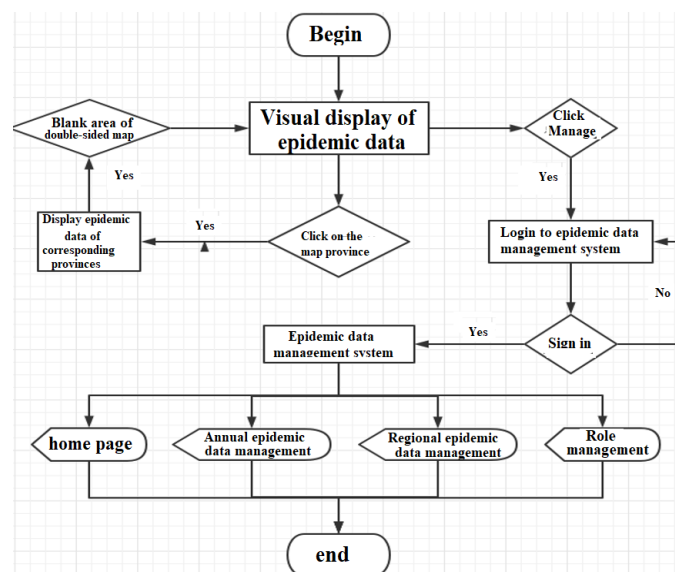


Figure 3. System Flow Chart

C. Database design

MySQL is used as the database, and the name of the database is eChart. One national epidemic summary data table, one User information table and 31 provincial data tables are established based on each province in China, as described and detailed in Table I below.

TABLE I. DESCRIPTION OF 33 TABLES IN THE DATABASE

Table Name	Table description
China	National epidemic summary data
User	User information table
Beijing	Epidemic Data Sheet of Beijing
...	...
Guangdong	Epidemic Data Sheet of Guangdong

Table II shows the field design of the national table and the 31 provincial tables. The national table includes the total number of outbreaks, the name of the province, the number of new cases, the existing number, the cumulative number, the number of cured cases and the number of deaths, which correspond to the year, month and current month respectively. Each of the 31 provincial tables contains the names of prefecture-level cities in each province, the number of new people, the existing number of people, the cumulative number of people, the number of cured people, and the total number of epidemics in each year, each month, and each current year and month corresponding to the death toll.

See Table III for the field design of the User Information Table, which mainly includes the user's name (login account), user's real name, user's telephone number, user's email address and user's login password.

TABLE II. DESIGN OF NATIONAL TABLE AND 31 PROVINCIAL TABLES

Field Name	Field description	Data type
Id (primary key, autoincrement)	Serial number	int
data_year	Year	varchar
data_month	Month	varchar
number	Number of people affected by the epidemic	int
province	Province/region	varchar
newnumber	Number of new people	int
nownumber	Current strength	int
totalnumber	Cumulative headcount	int
curennumber	Number of people cured	int
deathnumber	Death toll	int

TABLE III. DESIGN OF USER INFORMATION TABLE

Field Name	Field description	Data type
Id (primary key, autoincrement)	Serial number	int
user_id	User nickname	varchar
user_name	User's real name	varchar
user_phone	User phone number	varchar
user_email	User email address	varchar
user_password	User login password	varchar

D. Front-end and back-end separated development interface design

The front-end invokes the API interface to send a request, transmits parameters through the Axios technology, and returns data from the back-end database. The data interaction is shown in Figure 4 below.

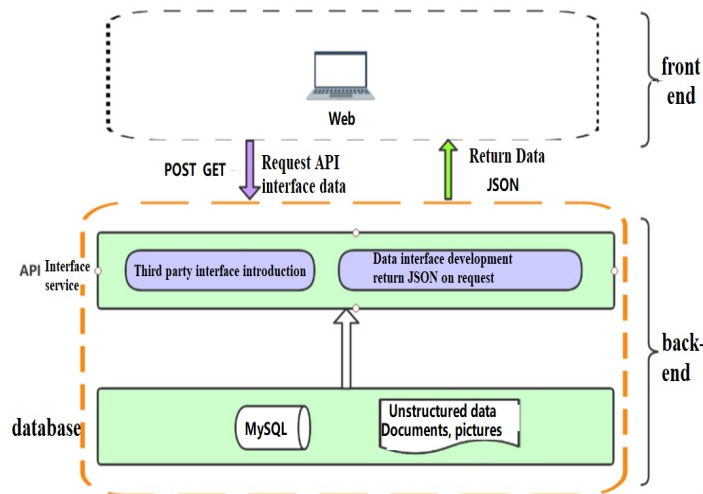


Figure 4. Data interaction diagram

Details are shown in Table IV below for the outbreak visualization backend API interface.

TABLE IV. BACKEND API INTERFACE CORRESPONDENCE TABLE

Interface function	An HTTP Request	URL	Support format
Get the number of cures and deaths across the country/region	GET	/all/{world}	JSON
Obtain annual epidemic data	GET	/ydata/{world}/{year}	JSON
Obtain existing and cumulative data	GET	/line/{world}	JSON
Get the top ten provinces/regions for new regions	GET	/bar/{world}	JSON
Obtain the number of cured people in the province/region	GET	/curebar/{world}	JSON
Get province/region name information	GET	/hotpie/{world}	JSON

III. SYSTEM REALIZATION

A. Data acquisition and storage

It adopts big data technology including collection, storage, analysis, presentation and other aspects. Crawler technology is used to collect data, and MYSQL is used to store and analyze the data. The system calls related data interfaces and encapsulates variables for method calling, obtains all data required by the epidemic situation visualization system through the URL data interface, stores the data in a database, crawls real-time epidemic situation data, obtains Response data through a Request request, groups the Response data to obtain epidemic situation cumulative data and real-time epidemic situation data, After each group of data is traversed and process, that data conforming to the database table structure is obtained and stored in the database [7] [8].

Operation steps:

- Connect to database `conn = pymysql. Connect ()`
- Generate cursor object `cursor = conn. Cursor ()`
- Execute SQL statement `cursor Execute(sql)`
- Close cursor `cursor Close()`
- Close the connection `conn. Close ()`
- Introducing navicat for data storage

Implementation code

```
db=pymysql.connect(host='localhost',user='root',password=' ',database='echart')
```

```
Cursor = db. Cursor () # Get a cursor
```

```
SQL = """ select * from China """ # Deal with domestic epidemic
```

```
Cursor. Execute (SQL)Result = cursor. Fetchall () # result is a tuple
```

```
# Save the tuple data in the list, 'province', 'cumulative number of people', 'number of deaths', 'number of cured ', 'number of existing ', 'cumulative confirmed increment ', 'death increment 'and' cure increment '. Existing confirmed increment "
```

```
Province = [] # province
```

```
Totalnumber = [] # Cumulative headcount
```

```
Deathnumber = [] # number of deaths
```

```
Curenumber = [] # Number of people cured
```

```
Nownumber = [] # Current headcount
```

```
Confirmed Relative = [] # Cumulative headcount
```

```
for i in result:
```

```
    province.append(i[0])
```

```
    confirmed.append(i[1])
```

```
    died.append(i[2])
```

```
    crued.append(i[3])
```

```
print(i)
```

B. Back-end implementation technology

Interface file (echart-server) source code

```
var express = require('express');
```

```
var mysql = require('mysql');
```

```
var app = express();
```

```
app.use(require('cors')());
```

```
app.use(express.json());
```

```
var db = mysql.createConnection({
```

```
    host:'localhost',
```

```
    user:'root',
```

```
    password:'',
```

```
    database:'echart',
```

```
    port:'3306'
```

```
});
```

```
db.connect((err)=>{
```

```
    if(err) throw err;
```

```
    console.log('success');
```

```
});
```

```
app.get('/api/all/:world',(req,res)=>{
```

```
    let sql = `SELECT SUM(curenumber) as allcurenumber,SUM(deathnumber) as allddeathnumber from ${req.params.world}`;
```

```
    db.query(sql,(err,result)=>{
```

```
        if(err){
```

```
            console.log(err);
```

```
        }else{
```

```
            console.log(result);
```

```
            res.json(result);
```

```
        }
```

```
    })
```

```
});
```

```
app.get('/api/allprovince',(req,res)=>{
```

```
    let sql = `SELECT province from China where province != 'Hong Kong' AND province != 'Macau' AND province != 'Shanghai';
```

```
    db.query(sql,(err,result)=>{
```

```
        if(err){
```

```
            console.log(err);
```

```
        }else{
```

```
            console.log(result);
```

```
            res.json(result);
```

```
        }
```

```
    })
```

```
});
```

C. Data Management

The Vue web page framework and the elementUI component library are used to establish the data management system, and the main interface is shown in Figure 5 below. The system calls the API interface through axios to manage the background data (such as addition, deletion, modification, and query).

D. Data visualization

The system visualization utilizes the ECharts charting technique, as shown in Figure 6. Use the Requests module to obtain the real-time epidemic data provided by the official website, and store the obtained structured data in MySQL. Through the time module of JavaScript, refresh the page regularly to obtain real-time data and improve the user experience. The data visualization code is as follows:

```
from pyecharts.charts import Bar
from pyecharts.faker import Faker
from pyecharts.globals import ThemeType
```

```
c=(Bar({"theme": ThemeType.MACARONS}))
.add_xaxis(china_data[40:])
.add_yaxis ( "" new confirmed "", newnumber)
.add_yaxis ( "" cumulative diagnosed "", totalnumber)
.reversal_axis()
.set_global_opts(
Title_opts = { "text": "new confirmed and cumulative
confirmed bar chart", "subtext": "epidemic big data" }
)
.set_series_opts(label_opts=opts.LabelOpts(position="
right"))
)
c.render_notebook()
```

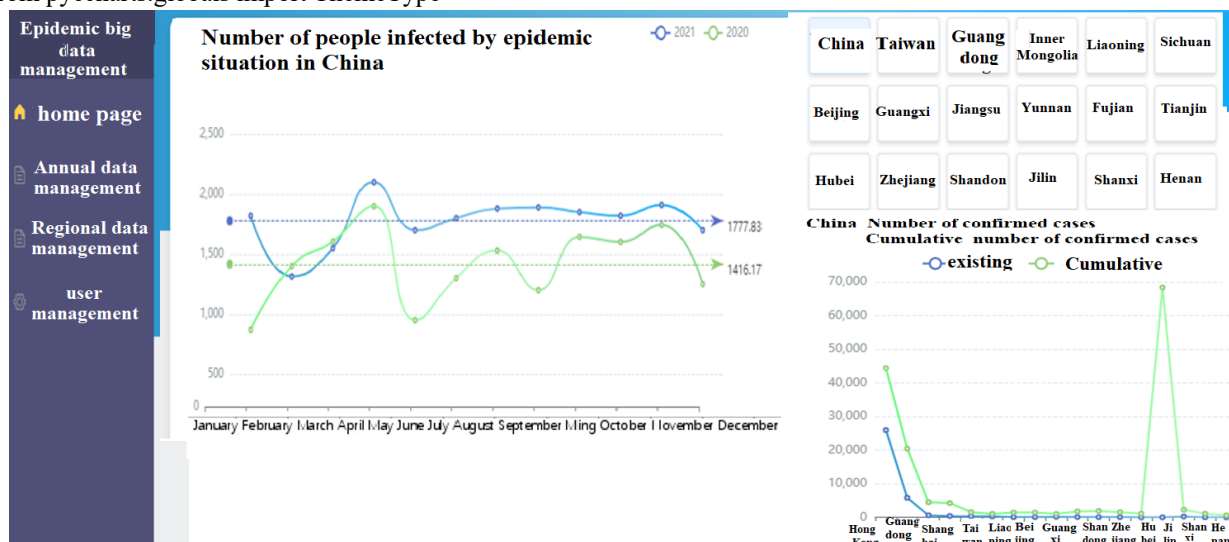


Figure 5. Epidemic data management system interface

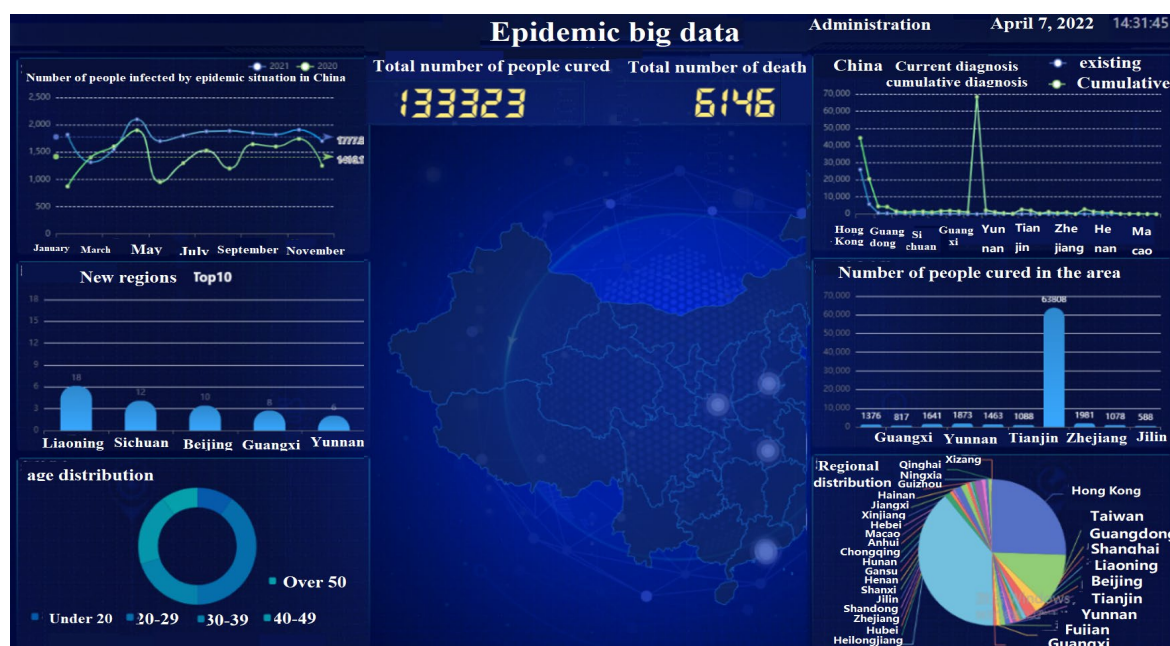


Figure 6. Display interface of national epidemic data

IV. CONCLUSIONS

The front-end of this project combines ECharts and elementUI component library, and the back-end is based on node.js Web application framework Express framework development. Using the combination of maps and visual graphics, with the help of the rich visual graphics library and map elements in Echarts technology, the epidemic analysis data are displayed in a diversified way, and the visualization of COVID-19 pandemic data is realized, which plays a vital role in improving residents 'self-protection awareness and doing a good job in epidemic prevention and control. The next step of this platform is to provide service support for more scenarios, and to provide scientific and intelligent processing for outbreaks by using big data, artificial intelligence and neural network technology.

ACKNOWLEDGMENT

Fundings: This work is supported by:

1) Research on Key Technologies and Applications of Smart Classroom Based on Big Data Analysis, a special project in the key field of colleges and universities of Guangdong Provincial Education Department in 2021 (Project No.2021ZDZX1127).Research and development projects in key areas of Guangdong Province(Project No: 2018B010109003).

2) 2022 School-level Teaching Reform Project of Guangdong Polytechnic of Science and Technology "Research on the Application of Online and Offline Blended Teaching Mode Based on Smart Classroom"(Project No.JG202230). School level scientific research project of Guangdong Polytechnic of Science and Technology (Project No.xjpy202104). Computer Vision Application Innovation Team(Project No: 2022KCXTD047)

3) Research on Evaluation System of Higher Vocational Colleges Based on big data. Scientific research project of

Guangdong Provincial Department of Education - characteristic innovation project (Project No.2021wtscx166).

4) Guangdong Provincial Education Science Planning (Higher Education Special), Educational Science Planning of Guangdong Province Research on personalized intelligent learning model driven by education big data and knowledge map - taking the training of higher vocational talents as an example. Research on Block chain Cloud Storage Model and Algorithm Based on Genetic Prediction Dynamic File, a special project in the key field of colleges and universities of Guangdong Provincial Education Department in 2020(Project No: 2020ZDZX3118).

REFERENCES

- [1] Chen Junsheng, Peng Lifen. Design and Implementation of Big Data Visualization System Based on Python + ECharts. Journal of Anhui Electronic Information Vocational and Technical College, 2019 (4): 6-9, 22.
- [2] XUE J K, SHEN B. A novel swarm intelligence optimization approach: sparrow search algorithm. Systems Science & Control Engineering, 2020, 8(1): 23-24.
- [3] Apache official website. ECharts feature. <https://ECharts.apache.org/zh/feature.html>, 2022.7.31.
- [4] Zhou Hongbin, Chen Liping, Liu Lianhao. Data visualization application based on ECharts. Journal of Shazhou Institute of Technology, 2021 (3): 3-9.
- [5] Xu Mengya. Design and Development of Enterprise Data Visualization Based on ECharts Technology. Modern Information Technology, 2022 (6): 90-96.
- [6] Guan Ziliang et al. Design and Implementation of Integrated Management Platform for Epidemic Prevention and Control. Computer Applications, No.7, 2022: 42-45.
- [7] Yang Chen et al. Design and Implementation of COVID-19 pandemic Big Data Visualization Platform. Computer Age 2021 Issue 1: 61-62.
- [8] Gu Rong. Big Data Processing Technology and System Research [D]. Nanjing University, 2016
- [9] Li Bo. Research and Implementation of Big Data Processing and Management Platform Based on Open Source Software [D]. Beijing University of Posts and Telecommunications, 2017.
- [10] Yang Xiaoxian. Thinking on the Application of Big Data Platform in Smart Education, Software, 2020, Vol. 41, No. 10: 270-271